Claude Code for Academic Researchers

Pre-Workshop Setup Guide

Instructor: Brady Allardice — brady.allardice@upf.edu

Please complete **all nine steps** below before the workshop. The whole process takes roughly **60–90 minutes of active work**, plus the large LaTeX download in Step 7 (which you can run in the background the day before) and two MCP installs in Steps 8–9 that are fiddly enough that they are best done calmly at home rather than in the room. If you get stuck on any step for more than 10 minutes, stop and email me at brady.allardice@upf.edu — don't try to debug it yourself. Once you are done, run the verification commands at the end and **send me a screenshot** of the output *and* your GitHub username (Step 2).

What You Need Before Starting

- A laptop (Mac or Windows) that you can install software on
- An internet connection
- A Claude Pro ($20/month) or Claude Max ($100/month) subscription — sign up at **claude.ai**
- A GitHub account and a free Zotero account — we will create both during the steps below if you don't already have them

The free Claude plan does NOT include Claude Code. You need at least a Pro subscription.

Step 1: Install VS Code

VS Code is the editor we will use throughout the workshop. It is free. If you already have it installed, make sure it is up to date (version 1.98 or newer).

- Go to **https://code.visualstudio.com**
- Download the version for your operating system (Mac or Windows)
- Install it and open it once to confirm it launches

Install the following VS Code extensions (click the Extensions icon in the left sidebar, or press Cmd+Shift+X on Mac / Ctrl+Shift+X on Windows, and search for each):

- **Python** (by Microsoft) — language support for Python
- **R** (by REditorSupport) — language support for R
- **Claude Code** (by Anthropic) — the official Claude Code extension

- **LaTeX Workshop** (by James Yu) — compiles LaTeX inside VS Code (we will install the TeX compiler itself in Step 7)

The Claude Code extension is the primary way you will interact with Claude Code during the workshop. It provides a chat panel inside VS Code where you can issue instructions and see file changes in real time.

Step 2: Install Git and Create a GitHub Account

Git is version control software that tracks changes to your files. We use it throughout the workshop, and Claude Code on Windows requires it to function. GitHub is a platform for hosting Git repositories; in Session 4 we run a **collaborative writing exercise** on a shared class repository, so I need to add each of you as a collaborator before then.

2a. Install Git

Mac: Open the Terminal app (search for "Terminal" in Spotlight) and type:

```
git --version
```

If Git is installed, you will see a version number. If not, your Mac will prompt you to install the Xcode Command Line Tools. Follow the prompt and wait for it to finish.

Windows:

- Go to **https://git-scm.com/downloads/win**
- Download and run the installer
- Use the default settings throughout the installer — do not change anything unless you know what you are doing
- When it asks about the default editor, you can select VS Code if you like

2b. Create a GitHub account (if you don't have one)

- Go to **https://github.com** and sign up for a free account

2c. Connect Git to GitHub

Open a terminal (Mac: Terminal app; Windows: open VS Code and use the integrated terminal via Ctrl+`) and run:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

Use the same email address you used to sign up for GitHub.

2d. Send me your GitHub username

So I can add you as a collaborator on the class repository before Session 4, **email me your GitHub username** (not your email address — the username, e.g. `octocat`) at brady.allardice@upf.edu. You can find it in the top-right of **https://github.com** after logging in.

Step 3: Install and Authenticate Claude Code

Claude Code comes in two interfaces: the VS Code extension you installed in Step 1, and a command-line tool. They are two front-ends to the same underlying program — you will primarily use the extension, but several setup commands (including the MCP installs in Steps 8–9) run from the command line, so we install and update both now.

3a. Install the command-line tool

Mac: Open the Terminal app and run:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Close and reopen your terminal after installation.

Windows: Open PowerShell (search for "PowerShell" in the Start menu) and run:

```
irm https://claude.ai/install.ps1 | iex
```

Close and reopen PowerShell after installation.

Do NOT run PowerShell as Administrator. Claude Code on Windows uses Git Bash internally, which is why Step 2 must be completed first.

3b. Update to the latest version

The MCP install commands in Steps 8–9 assume Claude Code 2.1 or newer. Update and confirm the version:

```
claude update
```

```
claude --version
```

Expected: `2.1.x` *or higher.*

3c. Authenticate the command-line tool

Open a new terminal and type:

```
claude
```

This will open your browser and ask you to sign in with your Claude account. Follow the prompts, then return to the terminal. You should see the Claude Code welcome screen. Type `/exit` to close Claude Code for now.

3d. Authenticate the VS Code extension

Open VS Code. Click the Claude Code icon in the left sidebar (the spark icon). It will prompt you to sign in. Follow the browser prompts — this uses the same account as Step 3c. Once authenticated, you should see a chat panel where you can type a message to Claude. Type something simple like "Hello" to confirm it responds.

Step 4: Clone the Course Repository and Set Up Your Workspace

All the exercises, demo data, slides, and the Zotero practice library (Step 9) live in a single GitHub repository. Cloning it now gives you a local copy to work from, and means you can pull updates before each session.

Open a terminal (Mac: Terminal; Windows: open VS Code and use the integrated terminal via Ctrl+`) and run:

```
cd ~ && git clone https://github.com/bradyallardice/claude-code-workshop.git
```

This creates a `claude-code-workshop/` folder in your home directory. Confirm it worked:

```
cd ~/claude-code-workshop && git status
```

Expected: `On branch student` *and* `nothing to commit, working tree clean.`

You can clone it somewhere other than your home folder if you prefer (e.g. Documents) — just remember where you put it and substitute that path below.

The per-session workspace workflow

From the workshop onward we use a deliberate split: the course repo stays **read-only** on your side, and you do all your actual work in a separate `~/my_workspace/` folder. This means I can push new material any time without breaking whatever you have in progress, and you can edit/break/delete freely without worrying about merge conflicts against the shared repo.

Run these three commands at the start of every session (shown here for Session 4 — substitute the current session's folder name):

```
# 1.  Pull the latest course repo
cd ~/claude-code-workshop && git pull

# 2.  Copy today's session folder into your workspace
mkdir -p ~/my_workspace && cp -r session_4/ ~/my_workspace/session_4/

# 3.  Open your copy in VS Code and work there
cd ~/my_workspace/session_4 && code .
```

From then on, **edit files in `~/my_workspace/`, not in the course repo itself.** If `git pull` ever fails with a merge conflict, that is exactly the problem we fix in class — don't try to resolve it. Just email me and I will help you reset cleanly.

Step 5: Install Python and Required Packages

The workshop exercises use Python. Even if you normally work in R or Stata, you will need Python installed because Claude Code will write and execute Python scripts during the exercises.

Install Python

- Go to **https://www.python.org/downloads/**
- Download Python 3.11 or 3.12 (either is fine)

Windows — important:

During installation, check the box that says "Add Python to PATH." If you miss this, nothing will work.

Mac: The installer is straightforward. Run it and follow the defaults.

Install required Python packages

After Python is installed, **close and reopen your terminal**, then run:

```
pip install pandas numpy statsmodels scipy linearmodels matplotlib seaborn scikit-learn
openpyxl
```

If you get an error saying `pip` is not found:

Mac:

```
pip3 install pandas numpy statsmodels scipy linearmodels matplotlib seaborn scikit-learn
openpyxl
```

Windows:

```
py -m pip install pandas numpy statsmodels scipy linearmodels matplotlib seaborn
scikit-learn openpyxl
```

If you already have Python installed via Anaconda, that is fine — you likely already have most of these packages (Anaconda does not always include `linearmodels`; if the verification command at the end complains about it, install it with `pip`, as above). You can skip the rest of this step but please verify by running the verification commands at the end.

Installing these packages globally, as above, is the baseline for this workshop. In Session 1 I will show you how to put them in a virtual environment (a separate, self-contained set of packages for one project), which you can do then if you prefer.

Step 6: Install R (if you don't already have it)

If you already have R installed, skip this step. If you are an R user but only have RStudio, you still need R itself installed so that the VS Code R extension and Claude Code can find it.

- Go to **https://cran.r-project.org**
- Download and install R for your operating system
- You do not need RStudio for this workshop, but you can keep it if you have it

Optional (R users): install the R packages the scripts use

Open R (or the R terminal in VS Code) and run:

```
install.packages(c("readr", "dplyr", "tidyr", "ggplot2", "MASS"))
```

The Session 5 capstone installs its own additional packages, including `cregg` version 0.4.0, the first time you run `session_5/capstone-template/reference_code/appendix.R`, so you do not need to install those now.

Optional (Stata users): install two user-written packages

Stata itself requires your own licence, which I cannot provide. If you plan to run the Stata (`.do`) exercises, install two user-written packages once, by typing the following inside Stata:

```
ssc install estout
ssc install outreg2
```

`outreg2` is used by `session_1/stata_demo/analysis.do`, and `estout` (which provides `esttab`) by `session_4/option_b/exercise_1/solution_robustness.do`.

Step 7: Install a TeX Distribution

In Session 3 we will write and compile a LaTeX paper directly inside VS Code, so Claude Code can edit, compile, and version-control your paper alongside your code and data. You already installed the **LaTeX Workshop** extension in Step 1 (the editor integration); now you need a TeX distribution (the actual compiler).

This is the largest download in the setup (2–4 GB) and can take 30–60 minutes. Do it the day before the workshop, not the morning of.

Mac — MacTeX:

- Go to **https://www.tug.org/mactex/**
- Download **MacTeX.pkg** (roughly 4 GB)
- Run the installer and follow the defaults

If you want a smaller install, **BasicTeX** (roughly 100 MB) works for this workshop — download it from the same page, then after installation run in Terminal:

```
sudo tlmgr update --self
sudo tlmgr install latexmk collection-fontsrecommended
```

Windows — TeX Live:

- Go to **https://www.tug.org/texlive/windows.html**
- Download **install-tl-windows.exe** and run it
- Use the default "full scheme" installation (roughly 2–4 hours to download; you can leave it running in the background)

If you are short on time or disk space, use the **MiKTeX** distribution instead (**https://miktex.org/download**) — it is smaller and installs missing packages on demand.

Verify it works

Open a terminal and run:

```
pdflatex --version
```

You should see a version number starting with `pdfTeX 3.x`. If the command is not found, close and reopen your terminal first. On Windows, you may need to log out and back in for the PATH to update.

Step 8: Install the GitHub MCP

An **MCP** (Model Context Protocol server) is how Claude Code talks to systems outside your project. The GitHub MCP lets Claude Code open pull requests, leave review comments, and create issues directly from the chat. We use it during the Session 4 collaborative story exercise. Setup takes 15–20 minutes and is too fiddly to do in the room.

8a. Create a GitHub Personal Access Token

A Personal Access Token (PAT) is a password-like string that lets the MCP act on your behalf. **Treat it like a password** — don't paste it into chat, don't commit it to git.

- Go to **https://github.com/settings/tokens?type=beta**
- Click **Generate new token**
- **Token name:** `claude-code-workshop`
- **Expiration:** 90 days
- **Repository access:** "All repositories" is fine for the workshop
- **Repository permissions:** grant **Read and write** on *Contents*, *Issues*, and *Pull requests*
- Click **Generate token**, then **copy the token immediately** — GitHub will not show it again

Paste it into a temporary scratch file you will delete after Step 8b.

8b. Add the GitHub MCP to Claude Code

In your terminal, paste the command below, **but before pressing Enter, replace the literal text `YOUR_PAT` with the token you just copied** (keep the word `Bearer` and the space before the token — you are only replacing `YOUR_PAT`, nothing else):

```
claude mcp add --scope user --transport http github https://api.githubcopilot.com/mcp/
-H "Authorization:  Bearer YOUR_PAT"
```

Example of what it should look like after substitution (with a fake token):

```
claude mcp add --scope user --transport http github https://api.githubcopilot.com/mcp/
-H "Authorization:  Bearer github_pat_11ABCDEFG0abc123XYZ..."
```

Do not drop `--scope user`. By default, `claude mcp add` registers a server only for the folder you ran it in ("local scope), so the server would vanish as soon as you open Claude Code in exttt extasciitilde/my_workspace/session_N. exttt--scope user registers it for your whole user account, so it is available in every folder.

Then verify:

```
claude mcp list
```

You should see `github` in the list with a ✓ `Connected` status. **Now delete the scratch file containing your token.**

8c. Confirm it works

Open Claude Code in any directory and ask:

```
Use the GitHub MCP to list the five most recent public repos owned by my account.
```

If it returns a list, you are done. If it asks you to authenticate or returns an error, see Troubleshooting at the end.

Step 9: Install the Zotero MCP

The Zotero MCP lets Claude Code search your Zotero library, pull BibTeX, and insert citations into a draft. We use it for a citations demo in Session 4 and you will use it for your own papers afterward. Setup takes 15–20 minutes.

9a. Set up Zotero and import the practice library

- If you don't have a Zotero account, create a free one at **https://www.zotero.org/user/register**
- Install the **Zotero desktop app** from **https://www.zotero.org/download/** if you don't already have it — the MCP needs the desktop app running to read your library (including full-text PDFs)
- Open the desktop app and sign in

Now import the practice library. I have prepared a bundle of 9 journal articles (with 10 PDF attachments — one article has two PDFs) for the in-class demo. After cloning the repo in Step 4, it lives at:

```
~/claude-code-workshop/session_4/docs/claude_code_zotero/
```

That folder contains a `claude_code_zotero.rdf` file and a `files/` subfolder of PDFs.

To import:

- In the Zotero desktop app, click **File** → **Import...**
- Choose **"A file (BibTeX, RIS, Zotero RDF, etc.)"**, click **Continue**
- Navigate to `session_4/docs/claude_code_zotero/` and select `claude_code_zotero.rdf`
- On the next screen, check **"Place imported collections and items into new collection"** and **"Copy files to the Zotero storage folder"**
- Click **Continue**

You should end up with a new collection called `claude_code_zotero` in the left sidebar containing 9 papers (top-level items), with 10 PDF attachments in total — one paper has two PDFs. **Confirm by clicking on the collection and verifying you see 9 papers, each with an attachment paperclip icon, and that expanding them shows 10 PDFs in total.**

Critical: keep the `files/` folder *next to* the `.rdf` on disk during import — if you move just the .rdf or import it from elsewhere, Zotero will import the metadata but not the PDFs.

9b. Let Claude Code talk to your Zotero app

Zotero blocks other applications from connecting by default. Turn that off:

- Open the Zotero desktop app
- **Mac:** **Zotero → Settings** **Windows:** **Edit → Settings**
- Go to the **Advanced** tab
- Under **General**, check the box **"Allow other applications on this computer to communicate with Zotero"**
- Close the Settings window. **Leave Zotero running** — the MCP needs it open to query your library

The MCP also falls back to the Zotero Web API if the desktop app is closed, but the local connection is faster and gives Claude Code access to your PDFs — worth turning on.

9c. Get your Zotero API key and Library ID

- Go to **https://www.zotero.org/settings/keys**
- Your **Library ID** (a number, e.g. `1234567`) is shown at the top under "Your userID for use in API calls"
- Click **Create new private key**
- **Key name:** `claude-code-workshop`
- Check **Allow library access** (read-only is fine; check write access too if you want Claude to add items)
- Click **Save Key**, then **copy the key immediately**

Save both the key and your Library ID into a temporary scratch file.

9d. Install the `zotero-mcp-server` package

Mac:

```
pip install zotero-mcp-server
```

If `pip` is not found, use:

```
pip3 install zotero-mcp-server
```

Windows:

```
py -m pip install zotero-mcp-server
```

Make sure you type `zotero-mcp-server` (not `zotero-mcp`) in the install command: a different, unrelated package on PyPI is called `zotero-mcp`. The package we use installs a command-line program called `zotero-mcp`.

Then confirm it installed:

```
zotero-mcp --help
```

Expected: a help message listing options. If you see "command not found," close and reopen your terminal first.

9e. Add the Zotero MCP to Claude Code

Run (substituting your real values from Step 9c):

```
claude mcp add --scope user zotero -e ZOTERO_LOCAL=true -e ZOTERO_LIBRARY_ID=YOUR_LIBRARY_ID -e
ZOTERO_API_KEY=YOUR_API_KEY -e ZOTERO_LIBRARY_TYPE=user - zotero-mcp serve
```

Then verify:

```
claude mcp list
```

You should now see both `github` and `zotero` listed with ✓ `Connected`. **Delete the scratch file with your key and Library ID.**

`ZOTERO_LOCAL=true` tells the MCP to read from the Zotero desktop app (the setting you turned on in Step 9b), which is what gives Claude Code access to your PDFs. The API key and Library ID are used for the Zotero web API, which the MCP falls back to if the desktop app is not running. **The web API only sees the practice collection once the Zotero desktop app has finished syncing it to your online library**, so after the import, leave Zotero open until the sync icon at the top right stops spinning. As in Step 8, `--scope user` is what makes the server available in every folder.

9f. Confirm it works

Open Claude Code in any directory and ask:

```
Use the Zotero MCP to list 5 items from the claude_code_zotero collection.
```

If it returns titles from the practice library (Alt et al., Bisgaard, Hicks et al., etc.), you are done. **Make sure the Zotero desktop app is open** when you run this — if it isn't, the MCP will either fail or fall back to the (slower) Web API.

Verification: Confirm Everything Works

Open a terminal (Mac: Terminal; Windows: PowerShell) and run each of the following commands. **Take a screenshot of the full output and send it to me**, along with your GitHub username from Step 2.

```
git --version
```

Expected: something like `git version 2.x.x`

```
python --version
```

Expected: `Python 3.11.x` *or* `3.12.x` *(Mac users: try* `python3 --version` *if this fails)*

```
python -c "import pandas; import numpy; import statsmodels; import scipy; import
linearmodels; print('All packages OK')"
```

Expected: `All packages OK` (Mac users: try `python3` instead of `python`)

```
claude --version
```

Expected: `2.1.x` or higher.

```
R --version
```

Expected: `R version 4.x.x` (skip this if you are not an R user)

```
pdflatex --version
```

Expected: `pdfTeX 3.x` followed by a TeX Live or MiKTeX version line

```
claude mcp list
```

Expected: both `github` and `zotero` listed with ✓ `Connected`.

```
zotero-mcp --help
```

Expected: a help message (any output is fine; "command not found" is the failure mode).

Also confirm:

- VS Code opens and the Python, R, Claude Code, and LaTeX Workshop extensions are visible in the Extensions sidebar
- The Claude Code extension panel responds when you type a message
- You have a GitHub account and have emailed me your GitHub username
- The `claude-code-workshop/` folder exists and `git status` inside it reports a clean working tree
- The `claude_code_zotero` collection in your Zotero app contains 9 papers with 10 PDF attachments in total (one paper has two PDFs)
- Asking Claude Code "list 5 items from the claude_code_zotero collection" returns titles
- Asking Claude Code "list my five most recent GitHub repos" returns repo names

One last thing for Session 4: it leans heavily on the Git workflow from Session 3, so in the few days before that class, spend five minutes re-reading `slides/session3_slides.pdf` in the course repo — specifically the `commit` → `instruct` → `diff` → `decide` cycle and how branches work.

Troubleshooting

"command not found" for any tool: Close your terminal completely and open a new one. If it still fails, the tool was not added to your PATH during installation. Email me with a screenshot of the error.

Claude Code authentication fails: Make sure you have an active Claude Pro or Max subscription at claude.ai. The free plan does not include Claude Code.

Python packages fail to install: Try running the pip command with `--user` at the end, e.g. `pip install pandas --user`. If you are on a managed or institutional machine, you may need to use a virtual environment — email me at brady.allardice@upf.edu and I will help.

Windows: Git not recognized after installing: Make sure you closed and reopened your terminal after installation. If it still fails, the Git installer may not have added Git to your PATH — reinstall and ensure "Git from the command line" is selected.

Windows: Claude Code install fails: Make sure Git for Windows is installed first (Step 2). Claude Code on Windows requires Git Bash internally.

LaTeX: `pdflatex` not found after installing: On Mac, log out and log back in — MacTeX updates PATH via a login item. On Windows, restart your computer after the TeX Live installer finishes.

LaTeX: installer is taking hours: That is normal — TeX Live's full scheme is several gigabytes. Let it run in the background. If it seems stuck, switch to MiKTeX (Windows) or BasicTeX (Mac).

`claude mcp add` fails with "unknown flag": You are on an older Claude Code. Run `claude update` (Step 3b) and try again.

GitHub MCP shows "Connected" but tool calls fail with 401/403: Your PAT is missing a permission. Regenerate it (Step 8a) and make sure *Contents*, *Issues*, and *Pull requests* are all set to *Read and write*. Then re-run the `claude mcp add` command — it will overwrite the existing entry.

Zotero MCP shows "Connected" but returns empty results: Your library is genuinely empty, or `ZOTERO_LIBRARY_TYPE` is wrong. If you set up a *group* library instead of a personal one, change `user` to `group` in the env var.

`zotero-mcp: command not found` after `pip install`: Close and reopen your terminal. If still failing, the install went to a directory not on your PATH — email me with the output of `pip show zotero-mcp-server` and I will help.

`claude mcp list` shows `github` or `zotero` in one folder but not in another: The server was registered with the default "local" scope, which only applies to the folder where you ran `claude mcp add`. Re-run the command with `--scope user` (as shown above). If Claude Code says the server already exists, first remove the old entry with `claude mcp remove github` (or `claude mcp remove zotero`) and then run the `claude mcp add` command again.

You accidentally pasted a token into a chat or committed it to git: Revoke it immediately (GitHub: **https://github.com/settings/tokens**; Zotero: **https://www.zotero.org/settings/keys**) and generate a new one. Tokens are easy to rotate; a leaked token in a public repo is not.

Anything else: Email me at brady.allardice@upf.edu. Do not spend more than 10 minutes troubleshooting on your own. I would much rather help you before the workshop than lose time during it.